

Calculate wp for Loop-Free Programs

- $wp(S, q) \equiv D(S) \wedge wlp(S, q) \wedge D(wlp(S, q))$
1. Let $w \Leftrightarrow wp(x := b[k], \text{sqrt}(x) \geq 1)$, calculate w .
 - $D(x := b[k]) \equiv D(b[k]) \Leftrightarrow 0 \leq k < \text{size}(b)$
 - $wlp(x := b[k], \text{sqrt}(x) \geq 1) \equiv \text{sqrt}(b[k]) \geq 1$
 - $D(wlp(x := b[k], \text{sqrt}(x) \geq 1)) \equiv D(\text{sqrt}(b[k]) \geq 1) \Leftrightarrow b[k] \geq 0 \wedge 0 \leq k < \text{size}(b)$
 - $$\begin{aligned} w &\equiv (0 \leq k < \text{size}(b)) \wedge (\text{sqrt}(b[k]) \geq 1) \wedge (b[k] \geq 0 \wedge 0 \leq k < \text{size}(b)) \\ &\Leftrightarrow \text{sqrt}(b[k]) \geq 1 \wedge b[k] \geq 0 \wedge 0 \leq k < \text{size}(b) \\ &\Leftrightarrow b[k] \geq 1 \wedge 0 \leq k < \text{size}(b) \end{aligned}$$
 2. Let $w \Leftrightarrow wp(x := y; z := v/x, z > x + 2)$, calculate w .
 - $wlp(x := y; z := v/x, z > x + 2) \equiv wlp(x := y, v/x > x + 2) \equiv v/y > y + 2$
 - $D(wlp(x := y; z := v/x, z > x + 2)) \equiv D(v/y > y + 2) \Leftrightarrow y \neq 0$
 - $$\begin{aligned} D(x := y; z := v/x) &\equiv D(x := y) \wedge wp(x := y, D(z := v/x)) \\ &\equiv T \wedge wp(x := y, x \neq 0) \\ &\equiv T \wedge D(x := y) \wedge wlp(x := y, x \neq 0) \wedge D(wlp(x := y, x \neq 0)) \\ &\equiv T \wedge T \wedge y \neq 0 \wedge D(y \neq 0) \Leftrightarrow y \neq 0 \end{aligned}$$
 - $w \equiv y \neq 0 \wedge v/y > y + 2 \wedge y \neq 0 \Leftrightarrow y \neq 0 \wedge v/y > y + 2$

Syntactic Substitution

- In backward assignment rule, we replace $Q(v)$ by $Q(e)$ to get the wlp for statement $v := e$ and postcondition $Q(v)$. The operation of going from $Q(v)$ to $Q(e)$ is called **syntactic substitution**. This substitution procedure is totally textual.

(Substitution in an Expression)

- **Substitution in an expression** is the following operation: take an expression e and replace its occurrence of variable v (of primitive types) with expression e' , written as $e[e'/v]$ and pronounced “ e with e' for v ”.
 - Substitution for a whole array is possible, for example: $b[x] [a / b] \equiv a[x]$, where a and b are both arrays.
 - Substitution at some index in an array is also possible, for example $b[x][b[x+2]/b[y]]$; this procedure can be tricky so we will discuss this more carefully in a future class.

- Like state updating, the squared parenthesis has very high hierarchy and when there are multiple substitutions, we calculate from left to right.

3. Finish the following substitution in expressions.

- $(x + y)[5 / x] \equiv 5 + y$
- $x + y [5 / x] \equiv x + y$
 - Substitution has very high hierarchy.
 - If there is no variable v in e , then $e[e' / v] \equiv e$.
- $(x + x)[2 / x] \equiv 2 + 2$
 - The substitution operation is done; and $(x + x)[2 / x] \not\equiv 4$.
- $(x * (x + 1)) [b - c / x] \equiv (b - c) * ((b - c) + 1)$
 - Add parentheses accordingly.
- if $x > 0$ then $-x$ else 0 fi $[z + 2 / x] \equiv$ if $z + 2 > 0$ then $-(z + 2)$ else 0 fi**

- In general, $e[e' / v]$ can be calculated using the following algorithm (assume v of primitive types):

$$e[e' / v] \equiv \begin{cases} c, & \text{if } e \text{ is a constant } c \\ e', & \text{if } e \equiv v \\ e, & \text{if } e \text{ is a variable and } e \not\equiv v \\ b[e_1[e' / v]][e_2[e' / v]] \dots [e_n[e' / v]], & \text{if } e \equiv b[e_1][e_2] \dots [e_n] \\ op(e_1[e' / v], e_2[e' / v], \dots, e_n[e' / v]), & \text{if } e \equiv op(e_1, e_2, \dots, e_n) \\ f(e_1[e' / v], e_2[e' / v], \dots, e_n[e' / v]), & \text{if } e \equiv f(e_1, e_2, \dots, e_n) \end{cases}$$

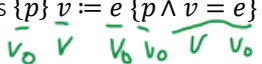
(Substitution in a predicate)

- **Substitution in a predicate** is the following operation: take a predicate p and replace each **free occurrence** of variable v with expression e' , written as $p[e' / v]$ and pronounced " p with e' for v ".
 - Again, the substitution is textual, so $(x > 0)[1 / x] \equiv 1 > 0$, but $(x > 0)[1 / x] \not\equiv T$.
 - If p is an unquantified predicate (in other words, p contains neither $\exists$ nor $\forall$), then we can calculate $p[e' / v]$ can using the algorithm we defined above for the calculation of expressions.
 - What if the predicate is quantified, or part of the predicate is quantified (such as $y < 0 \rightarrow \exists x. x \geq y$)? Let Q stand for a quantifier $\forall$ or $\exists$. In our intuition, $(Qx. q) [e / v]$ should be syntactically equivalent to $Qx. (q[e / v])$, but in fact this is not always true. We need to consider whether a variable is *free* or *bound*.
 - If an occurrence of a variable v in a predicate is within the scope of a quantifier over v , then it is a **bound occurrence**, else it is a **free occurrence**.
 - A variable v is **free** in p iff it has a free occurrence in p . Similarly, v is **bound** in p iff it has a bound occurrence in p .
4. Decide for each of the following variables, whether it is free or bound in p .
- $$p \equiv x > z \wedge \exists x. \exists y. y \leq f(x, y)$$
- Variable x is free and bound in p .
 - Variable y is bound in p .

- c. Variable z is free in p .
 - d. Variable u is neither free nor bound in p .
- From the above example we can see that each occurrence of some variable can be either free or bound. But for a variable, it can be both/either/neither free and/or/nor bound in a predicate.
- To syntactically substitute in a quantified predicate, we **only substitute for free occurrences**.
 - But there still are other problems. For example, if we have predicate $\exists x. x = v^2$ and we want to replace v by $y + 1$, then $(\exists x. x = v^2)[y + 1 / v] \equiv \exists x. x = (y + 1)^2$, and it looks correct to us. However, if we let $(\exists x. x = v^2)[x + 1 / v] \equiv \exists x. x = (x + 1)^2$, it totally changes the semantic of this predicate; we don't want to have such a substitution.
 - To $(Qx. q)[e / v]$, there are three different cases:
 - Case 1: $x \equiv v$.
 - Case 2: $x \not\equiv v$, and x does not appear in e .
 - Case 3: $x \not\equiv v$, and x appears in e .
 - For Case 1, we don't need to do anything, since we don't substitute for a bound occurrence: $(Qv. q)[e / v] \equiv Qv. q$.
 - For Case 2, we just need to substitute the free occurrences of v in q : $(Qx. q)[e / v] \equiv Qx. (q[e / v])$
5. Finish the following substitution in predicates.
- a. $(x > 0 \wedge \exists x. x \leq f(y))[17 / x] \equiv 17 > 0 \wedge \exists x. x \leq f(y)$.
 - b. $(y \geq 0 \rightarrow \forall x. x > y \rightarrow x * x > y \wedge \exists y. f(y) > x) [17 / y]$
 $\equiv 17 \geq 0 \rightarrow \forall x. x > 17 \rightarrow x * x > 17 \wedge \exists y. f(y) > x$
- For Case 3, we need to rename the quantified variable from x to something **fresh variable** (a variable that is not in e or q , such as z in this example): $(Qx. q)[e / v] \equiv (Qz. q[z / x]) [e / v] \equiv Qz. q[z / x][e / v]$.
 - A fresh variable is defined to be a variable that is not in e or q . However, to avoid unexpected problems, if possible, simply choose some variable that is not used anywhere in the whole predicate.
- For example: $(\exists x. x > 0 \rightarrow (\forall y. y + z < v^2) \wedge x > y)[y + 1 / v]$. When we only look at $(\forall y. y + z < v^2)[y + 1 / v]$, we are in case 3, and we can choose x as a fresh variable here to replace y ; but x is used in other places in the predicate. To avoid future ambiguity, it is better to use variable w here.
6. $((\exists x. x = v^2) \wedge h(y, v) > 0) [x + 1 / v]$
 $\equiv (\exists x. x = v^2) [x + 1 / v] \wedge (h(y, v) > 0)[x + 1 / v]$
 $\equiv (\exists z. (x = v^2)[z / x]) [x + 1 / v] \wedge (h(y, v) > 0)[x + 1 / v]$
 $\equiv (\exists z. z = v^2) [x + 1 / v] \wedge (h(y, v) > 0)[x + 1 / v]$
 $\equiv (\exists z. z = (x + 1)^2) \wedge h(y, x + 1) > 0$

Forward Assignment Rule

- Quick review, with partially valid triple $\{Q(e)\} v := e \{Q(v)\}$ we got the backward assignment rule:
 $wlp(v := e, Q(v)) \equiv Q(e)$.

- With the knowledge of syntactical substitution, we can generalize the **backward assignment rule** as follows:
 $wlp(v := e, q) \equiv q[e / v]$.
- Now, we have another question. What can be used as a postcondition for $\{p\} v := e \{?\}$ and what can be the strongest postcondition?
 - Intuition tells us $\{p\} v := e \{p \wedge v = e\}$ looks true, and it does work in some situations; but in general, it is not correct. 
- 7. Are the following triples valid (under either correctness)?
 - a. $\{x > y\} z := 2 \{x > y \wedge z = 2\}$.
 Yes. To justify if it is valid, we can apply the backward assignment rule to prove it:
 $wlp(z := 2, x > y \wedge z = 2) \equiv (x > y \wedge z = 2)[2 / z] \equiv x > y \wedge (2 = 2) \Leftrightarrow x > y$
 - b. $\{x > 0\} x := x - 2 \{x > 0 \wedge x = x - 2\}$.
 Clearly this is not valid, since $(x = x - 2) \Leftrightarrow F$; and we can see many states don't satisfy it.
 - Why $\{p\} v := e \{p \wedge v = e\}$ works in example 7.a but not 7.b? The difference is **whether v has free occurrence in e or p** : when v appears in e and p , it represents the “old value” before updating.
 - Here is how we solve this problem: we record the old value of v in precondition with a **logical constant** (which is a constant that only appears in pre- and/or post- conditions that helps us to reason):
 $\{x > 0 \wedge x = x_0\} x := x - 2 \{x_0 > 0 \wedge x = x_0 - 2\}$. We say that we **age** the variable x in the precondition.
- **Aging** a variable x is to introduce in the precondition a logical constant to name the value of x before executing the statement.